

CoCoTree: A Computation-Capable Architecture for Collective Communication in Scalable PIM

Shunchen Shi^{1,2,†}, Qijia Yang^{1,2,†}, Fan Yang^{1,2}, Yu Huang³, Youwei Zhuo⁴,
Zhichun Li^{1,2}, Ninghui Sun^{1,2}, and Xueqi Li^{1,2,*}

¹State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences

²University of Chinese Academy of Sciences

³Huazhong University of Science and Technology,

⁴Peking University

{shishunchen22z, yangqijia25e, yangfan24s}@ict.ac.cn, yuh@hust.edu.cn,
youwei@pku.edu.cn, {lizhichun23z, snh, lixueqi}@ict.ac.cn

Abstract—The growing demand for high-bandwidth and large-capacity memory access in data-intensive workloads has driven the development and deployment of Processing-in-Memory (PIM) architectures. However, existing DIMM-based PIM systems suffer from the severe communication bottleneck between the processing elements (PEs) near the PIM banks due to their requirement on host CPU forwarding. This bottleneck limits the efficiency of collective operations and degrades scalability and performance for workloads that require inter-PE communication.

To address the communication limitation, we propose CoCoTree, a computation-capable architecture for collective communication in scalable DIMM-based PIM. CoCoTree supports direct and high-throughput inter-PE communication without host intervention. CoCoTree accelerates key collective communication using novel hierarchical binary tree topology and lightweight in-network computation support. We design and implement microarchitectures for the main building blocks: Co-Leaf and Co-Node, to efficiently handle the data packing, routing, and processing in CoCoTree. Furthermore, we also introduce a packet-based communication protocol tailored to the CoCoTree architecture, which decouples control and data through a two-phase configuration-computation communication mechanism to efficiently support a wide range of collective communication operations. CoCoTree effectively mitigates inter-PE communication bottlenecks, enabling scalable PIM systems capable of meeting the demands of growing data size. Experimental results show that CoCoTree achieves up to 95.6 $\times$ improvement for collective operations and improves end-to-end application performance by up to 10.5 $\times$ across various workloads over the baseline PIM, while outperforming state-of-the-art PIM communication architectures in both performance and scalability.

I. INTRODUCTION

Emerging data-intensive workloads such as graph processing [1], [2], [5], [36], [66], bioinformatics [4], [6], [26], [83], large language model (LLM) [38], [42], [47], [48], [56], [62], [69] and personalized recommendation [16], [17], [50] impose significant requirements of high capacity and high bandwidth on memory systems. However, constrained by pin count limitation and power consumption, traditional memory systems struggle to satisfy growing performance requirements. This fundamental limitation stems from the massive data

movement bottlenecks between processors and memory, which constrains the overall system performance [22], [32].

Processing-in-Memory (PIM) [64], [65] architecture is a promising solution that integrates compute logic near the data within memory chips. This approach effectively eliminates data movement bottlenecks through near data processing, enabling high internal bandwidth and massive parallelism. Recently, researchers have proposed various processing-in-memory architectures to benefit data-intensive workloads like machine learning [10], [39]–[41], database operations [10], [51], [59], [74], sparse linear algebra [35], [36], recommendation systems [44], [45], [49], and genome analysis [6], [25]. Dual In-line Memory Module (DIMM) PIM architecture is an economical and practical approach, offering larger capacity, simplified manufacturing feasibility, and higher device compatibility compared to HBM-based solutions [50].

Despite their promise, current DIMM PIM architectures face significant limitations in processing element (PE) interconnectivity. Most existing systems lack direct communication paths between PEs. The predominant approach utilizes the CPU-forwarding mechanism for inter-PE communication, requiring all data transfers between PEs to be processed and forwarded through the host CPU [7], [27], [37], [68]. Previous studies [37], [76], [86] have demonstrated this approach routes all data exchange through the host CPU, which often restricts the inter-PE bandwidth to tens of gigabytes. This inter-PE communication bandwidth represents merely 2% of the aggregate bandwidth on existing PIM hardware [37], creating a communication bottleneck. This communication inefficiency particularly limits the performance of workloads requiring frequent collective operations (e.g., graph algorithms, machine learning), which exhibit communication patterns including *broadcast*, *Reduce-Scatter*, and *All-Reduce*, etc.

Such inefficient CPU-forwarding mechanisms fundamentally constrain the communication of PIM architectures [77]. Previous studies [19], [77] implement broadcast-based direct inter-PE communication through multi-drop bus structures, but face practical implementation challenges on DDR4/5 due to timing constraints and signal integrity issues [86]. DIMM-Link [86] employs dedicated links to support packet-based inter-

[†] Both authors contributed equally to this research

* Corresponding author

DIMM communication, and PIMnet [76] proposes a multi-tier network architecture for inter-PE communication in the single channel. However, they also face scalability limitations and still involve CPU forwarding. Moreover, existing optimization efforts for inter-PE communication predominantly target basic schemes like point-to-point and broadcast, overlooking the overhead of common communication patterns like All-Reduce in these workloads, while lacking specialized optimizations for these collective operations. Although some software collective communication frameworks [14], [68] offer flexible APIs, the lack of hardware support confines them to CPU-forwarding mechanisms. Consequently, inter-PE communication has become a primary bottleneck for PIM performance and scalability. Ideally, parallel systems should sustain their efficiency as the number of PEs increases, but in the current PIM architecture, IPC overhead grows with PIM system scale, diminishing the performance gains from additional PEs.

We observe that current inter-PE communication mechanism for DIMM PIM architectures still faces three key challenges: the first is the redundant data movement and excessive host CPU overhead in inter-PE communication, the second is constrained bandwidth scalability of collective communication, and the third is the lack of specialized hardware optimizations for common collective operations.

To tackle these challenges, we propose **CoCoTree**, a computation-capable architecture for the collective communication in scalable DIMM-based PIM systems. CoCoTree enables efficient direct inter-PE communication by offloading collective communication from the host CPU to a tree communication network with hardware support for in-network computation. CoCoTree consists of two primary components: Co-Node, which performs routing and computing operations in collective operations, and Co-Leaf, which acts as the interface between PEs and the network and implements data packing to align the data with the communication protocol. We also propose a packet-based communication protocol tailored to the CoCoTree architecture, which decouples control and data through a configuration-computation communication model to support a wide range of collective operations. In summary, we make the following contributions:

- We identify that Inter-PE Communication (IPC) bottlenecks limit the scalability of the current DIMM PIM architecture.
- We propose CoCoTree, a novel and scalable tree-based communication network architecture for DIMM PIM systems. CoCoTree supports efficient routing and in-network computation, enabling high-performance execution of collective communication operations and effectively addressing the IPC bottleneck in PIM.
- We design a packet-based communication protocol tailored for CoCoTree, which decouples control and data transfer/processing. By leveraging the regularity of a perfect binary tree, the protocol supports various collective communication patterns.
- We evaluate CoCoTree and experimental results show that CoCoTree achieves up to 95.6× speedup in collective

communication operations such as all-reduce compared to host-CPU forwarding, up to 10.5× workload acceleration, and 1.4× to 1.7× improvements over state-of-the-art PIM communication architectures.

II. BACKGROUND

A. DIMM PIM Architectures

Among various PIM solutions [24], [46], [53], [54], DIMM-based PIM architectures have gained significant attention in both academia and industry due to higher economic advantage and manufacturing feasibility [7], [62]. Compared to alternatives like HBM [54], DIMM-based solutions offer larger memory capacities, lower fabrication costs, and enhanced device compatibility. UPMEM PIM [24] is the first commercially available commodity processing-in-memory architecture. As illustrated in Figure 1, each UPMEM PIM DIMM adopts the form of a standard DDR4-2400 DIMM module. A PIM-server with UPMEM features a hierarchical architecture where each system can integrate up to 20 UPMEM DIMMs, with each DIMM containing two ranks of eight PIM chips. Every PIM chip incorporates eight independent 64MB DRAM banks, each directly coupled with a programmable DRAM processing unit (DPU). The DPU is a 32-bit scalar in-order core based on the RISC ISA. There are also two scratchpads, a 24KB instruction memory (IRAM) for program binary and a 64KB working memory (WRAM) for fast data access [24]. This DIMM PIM architecture is conducive to data-intensive workloads. The primary reason is the integration of DPUs onto DRAM chips, which facilitates the replacement of off-chip data transfers with low-power on-chip data movements.

However, as illustrated in Figure 1, a critical limitation exists in current DIMM-based PIM systems: there is no direct communication method between different PIM banks. Whether the communication occurs across banks within the same chip, between chips, or across ranks, all inter-PE data exchanges must be routed through the host CPU. Constrained by the narrow bandwidth of the memory channel between the host and DIMM PIM modules, this naive CPU forwarding mechanism limits the communication throughput and scalability of the PIM system. As a result, inter-PE communication has become a major bottleneck in DIMM PIM architectures.

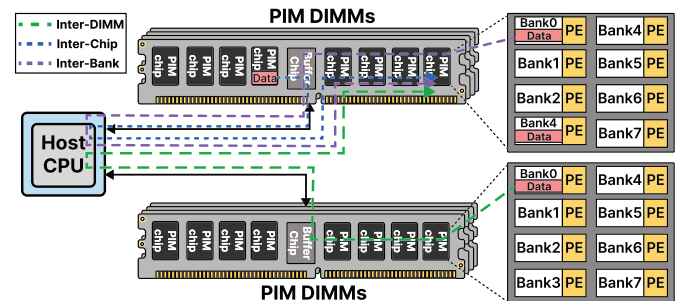


Fig. 1. System-level overview of communication in DIMM-based PIM architectures. Existing systems lack direct inter-PE communication paths. Data exchanges across banks, chips, or ranks require host CPU forwarding.

B. Collective Communication on DIMM PIM architecture

Collective communication is a fundamental mechanism for enabling efficient coordination in parallel computing systems. It typically involves orchestrated data exchanges and operations across multiple compute nodes, known as collective operations. Representative examples include Broadcast, All-Gather, All-Reduce, and Reduce-Scatter. For instance, an All-Reduce performs a reduction (e.g., sum) across distributed partial results from multiple PEs. These primitives allow PEs to efficiently coordinate complex data exchanges and computations, and are widely utilized in domains such as high-performance computing (HPC) [30], [55], [71], [85], machine learning [31], [73], [82], and graph processing [33], [34], [87]. As the number of PEs scales, the efficiency of collective communication becomes a key determinant of overall system performance and scalability.

While collective communication has been optimized for traditional distributed systems [9], [13], [78], these designs are generally based on the systems with hardware and protocol support [18], [20], [52], [72], making them not practical for PIM systems. Existing DIMM-based PIM architectures rely on host CPU forwarding to perform collective communication. Although this approach is simple to implement, it incurs frequent data transfers between PEs and the host CPU, resulting in significant overhead. This severely limits both the effective communication bandwidth and scalability of the system.

Recent research has explored hardware-based optimizations to mitigate these issues, such as introducing dedicated buses [77] or dedicated point-to-point links [86] to enable direct inter-PE communication. However, dedicated buses face signal integrity and timing challenges [86], which hinder their practicality. Similarly, dedicated links often suffer from limited scalability [19], [77] and still depend partially on CPU forwarding [76], [79], failing to completely eliminate CPU intervention. Therefore, enabling efficient, low-overhead, and scalable direct collective communication among PEs remains a critical challenge in current DIMM PIM architectures.

III. MOTIVATION

A. The Scaling Bottleneck of Inter-PE Communication

Although Processing-in-Memory (PIM) architectures offer significant acceleration for memory-bound applications by integrating compute capabilities into memory, commercial PIM systems (e.g., UPMEM) still suffer from limited scalability. This limitation stems primarily from the lack of efficient inter-PE communication support. To assess the severity of this bottleneck, we profile several memory-intensive benchmarks on a commercially available UPMEM platform. These workloads exhibit low compute-to-memory ratios and are ideal candidates for PIM acceleration. However, their execution involves substantial communication overhead—both between the host and the PEs, and among PEs themselves.

Figure 2(a) illustrates the detailed execution breakdown across varying PE counts for Breadth-First Search (BFS) [37] on the UPMEM. Inter-PE communication accounts for

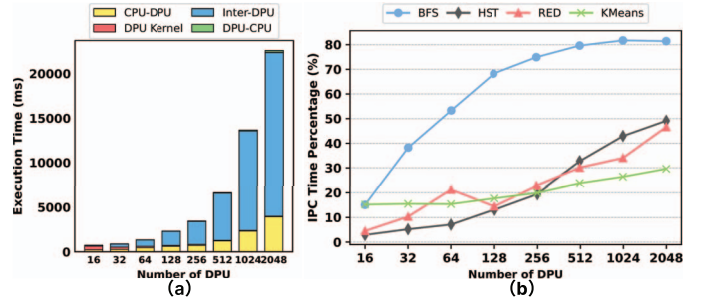


Fig. 2. (a) Execution time breakdown of BFS and (b) performance scaling on UPMEM across increasing PE counts.

approximately 15.1% of total execution time at 16 PEs. This part surges to 81.7% when scaling to 1024 PEs and maintains this proportion as the number of PEs increases. For some other applications [37], as the number of PEs increases, communication overhead significantly increases and dominates execution time. This trend highlights the bottleneck introduced by the CPU-forwarding communication model. The inter-PE communication through CPU-forwarding is limited by the narrow bandwidth between the host and DIMM PIM, undermining the potential benefits of massive on-chip parallelism and high internal bandwidth in PIM. This mismatch between PIM parallel compute capabilities and its communication infrastructure necessitates a scalable, low-latency communication architecture tailored to inter-PE communication.

B. Analysis of Existing Inter-PE Communication Methods

As shown in Figure 3, existing inter-PE communication mechanisms in DIMM PIM architectures can be categorized into three distinct approaches: CPU forwarding, dedicated bus, and dedicated link. Each approach exhibits fundamental limitations that constrain system scalability and performance.

(1) **CPU Forwarding:** CPU forwarding represents the predominant approach, where all inter-PE data transfers are processed and arbitrated through the host CPU. This approach incurs redundant PE-CPU-PE data movement and excessive host CPU overhead. The communication bandwidth is fundamentally constrained by the processing capability and memory bandwidth of the host CPU. While recent work [14] has proposed software-level communication API interfaces to improve efficiency, these approaches are still limited by host CPU bandwidth constraints. (2) **Dedicated bus architectures.** Previous work [19], [77] adopts multi-drop bus architectures to implement broadcast-based direct inter-PE communication. While these approaches reduce CPU intervention, they face significant practical implementation challenges, including timing constraints, signal integrity, and limited scalability due to electrical loading effects [86]. The broadcast-only nature also restricts support for other collective operations. (3) **Dedicated link interconnections.** Recent research [76], [79], [86] designs dedicated physical links to support communication between PEs. However, [86] shows limited scalability due to high latency and network congestion

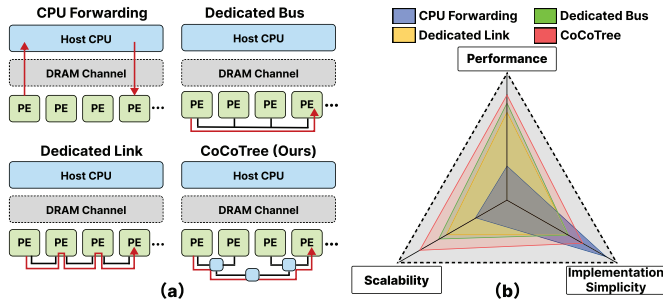


Fig. 3. (a) Current inter-PE communication mechanisms in DIMM PIM and (b) qualitative comparison of them.

under high communication loads, and the absence of dedicated hardware support for collective operations further limits their effectiveness. Additionally, [79] proposes a hardware-based bridge that does not provide direct bank-to-bank communication and still needs host CPU forwarding. PIMnet [76] provides a multi-tier interconnect but limits to 256 DPUs within a single memory channel. When DPUs across different channels require communication, PIMnet still relies on host forwarding, ultimately restricting system-level scalability.

These limitations highlight the urgent need for a scalable, low-overhead, collective communication architecture to eliminate host intervention and support efficient coordination among thousands of PEs within and across DIMM modules.

IV. CoCoTree ARCHITECTURE OVERVIEW

A. Overview

As illustrated in Figure 4, our proposed scalable PIM architecture, named CoCoTree, leverages a hierarchical binary-tree structure to address the inter-PE communication bandwidth limitations in traditional DIMM-based PIM systems.

CoCoTree System Architecture: The system consists of the DIMM PIM and our tree-structured collective communication network, CoCoTree. In this work, we take UPMEM as a representative DIMM PIM platform. PIM processing elements (PEs) execute data-intensive computational tasks near memory, whereas the CoCoTree supports efficient collective communications among these PEs.

CoCoTree Network: The CoCoTree can be conceptually viewed as a perfect binary tree composed primarily of two types of components: ① Co-Node and ② Co-Leaf. Integrated within each bank-level PIM PE, Co-Leaf connects PIM PE to the CoCoTree network. Co-Nodes function as intermediate nodes performing data processing and forwarding operations. Specifically, data from PEs enters the CoCoTree through the Co-Leaf and ascends hierarchically through the network. Each Co-Node performs configured processing or forwarding until the data reaches the root node of the working subtree. Subsequently, the processed data is disseminated back down the tree to the target PEs to complete the collective communication.

The placement of Co-Node and Co-Leaf is strategically designed to maximize system performance and minimize wiring complexity. Co-Leaves are embedded inside each PIM

PE for efficient data packing and unpacking, while Co-Nodes are arranged to form intra-chip binary trees. These chip-level trees are then connected via bi-directional SerDes links on the PCB to a dedicated rank-level CoCoTree, which can further be composed across multiple ranks into a DIMM-level CoCoTree, forming a hierarchical communication network for the PIM memory.

Co-Node: Each Co-Node is a lightweight, computation-capable, and configurable network switch with three interfaces connecting to parent, left-child, and right-child nodes. Co-Nodes receive upstream data from child nodes and execute reduction or forwarding based on configured settings, forwarding results upwards to parent nodes. Additionally, they manage downstream data received from parent nodes, selectively routing this data to the left, right, or both child nodes. Co-Nodes feature multiple Functional Units (FUs) to efficiently support low-overhead integer reductions of arbitrary byte-width and dynamically expandable data widths, especially beneficial for reduction operations.

Co-Leaf: The Co-Leaf functions as a bridge between PEs and Co-Nodes. It packs data from PEs into data packets that are compatible with Co-Nodes and transmits them accordingly. Co-Leaf also receives packets from Co-Nodes, unpacks and returns data back to the PE. In this work, we enhance existing PIM PEs by designating Co-Leaf as an additional target for DMA engines, enabling efficient PE-to-Co-Leaf data transfers. Inside each Co-Leaf, a packing unit and an unpacking unit are orchestrated by a local controller to provide efficient data packing and unpacking.

CoCoTree Communication Mechanism: CoCoTree adopts a lightweight packet-based communication mechanism. A two-phase communication model is proposed to decouple control and data flow. CoCoTree utilizes data and command packets for computation and configuration phases, respectively. Detailed designs of the packet format and stream control are elaborated in subsequent sections.

B. Collective Operation Implementation

CoCoTree supports a variety of collective communication operations, including broadcast, all-gather, all-reduce, and reduce-scatter, etc. These operations, which are essential in PIM systems, can be offloaded to the CoCoTree architecture and are efficiently executed through hierarchical routing and in-network computation within Co-Nodes, as well as data reassembly logic within Co-Leaves.

Point-to-point, multicast, and broadcast communications are all implemented via tree-based forwarding mechanisms within Co-Nodes, determined by the destination address. The primary difference among them lies in the routing strategy configured during the setup phase. In broadcast and multicast, data from the parent node is simultaneously forwarded to both child nodes, enabling efficient one-to-many distribution. In contrast, point-to-point communication selectively forwards data along a single path based on the encoded address. The all-gather operation is implemented as a pipelined series of broadcast stages, where each PE sequentially contributes its data to

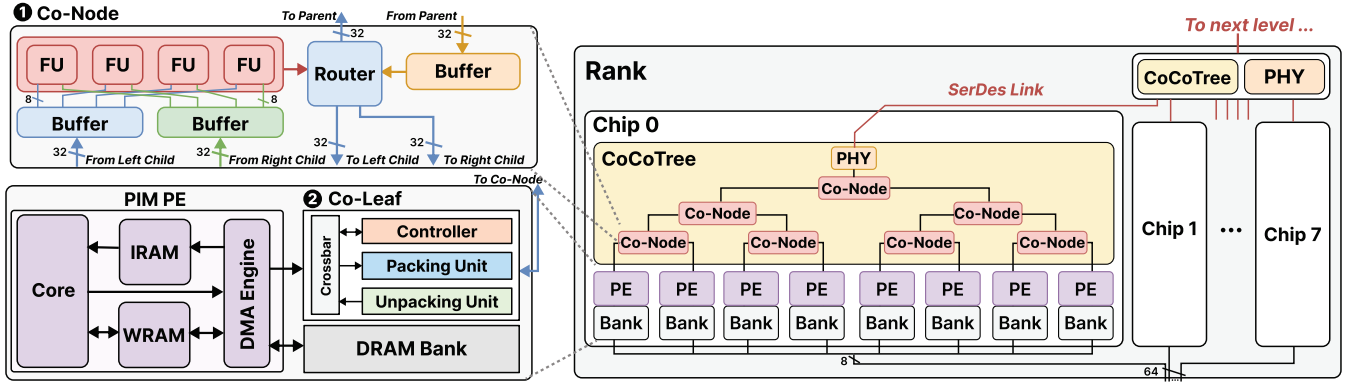


Fig. 4. CoCoTree system architecture overview. It consists of ❶ Co-Nodes for in-network computation and hierarchical data routing and ❷ Co-Leafs embedded in each PE for packetization.

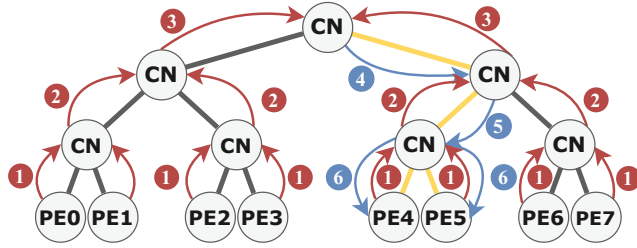


Fig. 5. Illustration of reduce operation using CoCoTree. (CN: Co-Node)

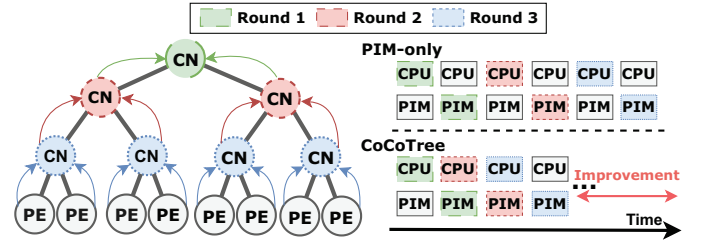


Fig. 6. Pipeline parallelism across Co-Nodes (CN), Host CPU and PIM.

the subtree. This design avoids bandwidth contention and ensures scalability. Reduce, all-reduce, and reduce-scatter are implemented via tree-based reductions, leveraging the built-in functional units (FUs) in Co-Nodes. As shown in Table I, supported operations include sum, bitwise and/or/xor and unsigned min/max. All operations support arbitrary integer widths. The all-reduce operation first performs an upward reduction, then broadcasts the result downward. The reduce-scatter operation is realized as a multi-stage all-reduce with selective filtering at Co-Leafs to partition the final result.

Figure 5 illustrates a reduce example on CoCoTree, where data from 8 PEs is reduced (Steps ❶-❷-❸) and broadcast to the target subtree (Steps ❹-❺-❻).

C. Pipelining

To further improve system performance and reduce the latency of collective communications, CoCoTree adopts a pipeline execution strategy that improves resource utilization across multiple communication rounds. This pipelining allows overlapping between computation and communication stages, enabling high throughput communication.

As illustrated in Figure 6, CoCoTree supports pipeline execution across Co-Nodes, allowing multiple collective communication operations to proceed concurrently at different levels of the hierarchy in the tree structure. Specifically, while the result of a previous communication round is still propagating through the upper levels of the tree, the next round can begin execution in a lower subtree. This hierarchical

pipeline within the CoCoTree improves utilization of internal network bandwidth and maximizes throughput. Figure 6 highlights the pipeline parallelism between the host CPU and the CoCoTree interconnect, demonstrating the performance advantage over traditional PIM systems. In conventional PIM architectures, each collective operation requires host CPU forwarding, leading to serialization between CPU scheduling and PIM execution. In contrast, CoCoTree offloads collective communication to the CoCoTree, eliminating the need for host intervention. This allows the host CPU to prepare data for subsequent tasks while collective operations are still executing in the network, enabling parallel execution across CPU and PIM domains and improving overall system efficiency.

D. Programming interface

To facilitate the utilization of CoCoTree, we introduce a flexible and user-friendly API for PIM-side kernels, enabling direct inter-PE collective communication. As illustrated in Figure 7(a)(b), we take the code segment for reducing the node bitmap `frontier` in Breadth-First Search (BFS) as an example to show the programming of CoCoTree. CoCoTree code is a part of the PIM kernel code, which is stored in the instruction scratchpad for each PE as shown in Figure 7(c). The API operates on a two-phase model: configuration and execution. First, a designated PE (e.g., PE#0 in the example) initiates the configuration phase by defining the parameters like the number of PEs and the operation type (e.g., `ReduceOR`), using `CoCoTree::initConfig()`. This configuration is then broadcast to the relevant PEs via

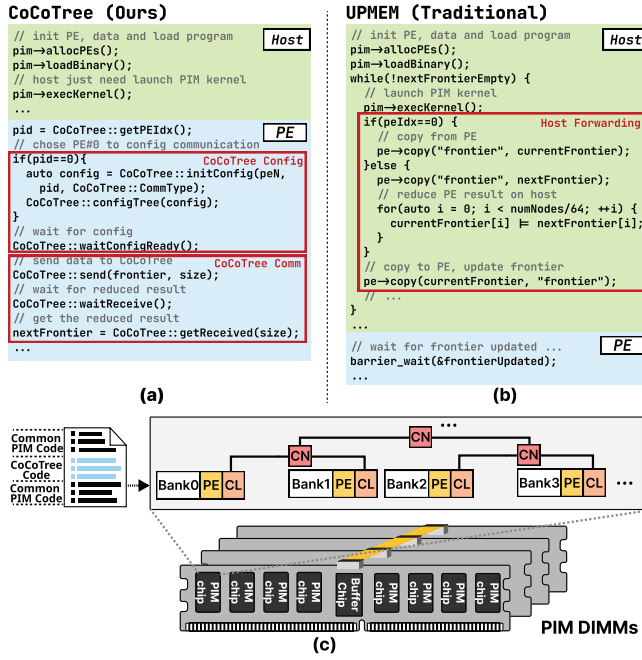


Fig. 7. (a) Code implementation in CoCoTree, with green and blue blocks denoting host-side and PE-side code, respectively. (b) Code implementation in conventional UPMEM. (c) The host pre-loads code of PIM kernel (including CoCoTree code) and data into each bank, like UPMEM.

CoCoTree::configTree(). All PEs synchronize at a barrier, CoCoTree::waitConfigReady(), to ensure the hardware is configured before proceeding. Once configured, the execution phase begins. Each participating PE injects its local data into the network using CoCoTree::send(). CoCoTree performs the specified in-network computation (e.g., a bitwise OR reduction for a parallel BFS frontier). PEs then call CoCoTree::waitReceive() to wait for the completion, after which the final result can be retrieved via CoCoTree::getReceived().

Traditional PIM programming models implement such collective operations through a host-centric control flow: the host CPU orchestrates explicit DMA transfers to gather data, performs the required processing, and then transfers the results back to target PEs. It increases code complexity on both the host and PIM sides and shows poor performance on inter-PE communication. With CoCoTree, the programmer expresses the entire collective directly inside the PIM kernel using CoCoTree APIs, while the CoCoTree hardware handles routing, synchronization, and in-network computation.

V. COCOTREE MICROARCHITECTURE

In this section, we introduce the microarchitecture design of CoCoTree. The architecture of CoCoTree is composed of two key components: the *Co-Leaf* and the *Co-Node*.

A. Co-Leaf Microarchitecture Design

The **Co-Leaf** serves as the interface between each DIMM PIM bank-level processing element (PE) and the CoCoTree

interconnect network. Co-Leaf dynamically packs and unpacks data while aligning the data layout with the tree-based communication protocol. Prior to transmission, the Co-Leaf reorganizes output data from its local PE at the byte granularity into packets compatible with Co-Nodes. This process disassembles each multi-byte value into individual bytes and then re-groups them to form structured packets. The byte reordering strategy is operation-specific: for *min/max* reductions, bytes are arranged in MSB first order to support unsigned element-wise comparisons; for arithmetic operations, LSB first ordering is used to preserve carry propagation across byte boundaries.

As shown in Figure 8(c), the packing and unpacking logic is implemented in hardware through a Packing Unit (1) and an Unpacking Unit (1), both managed by the central controller (2) within the Co-Leaf, handling bidirectional data flow. The PIM PE can program this controller via memory-mapped I/O (MMIO). Within the Packing/Unpacking Unit, a demultiplexer (4) distributes each incoming multi-byte word into individual FIFO buffers (3). Once all buffers are full, a concatenator (5) simultaneously dequeues one byte from each FIFO and merges them to form a packed packet. The Handshake Controller (6) then handles the transmission of packed data to Co-Node. Upon receiving data streams from Co-Node, CoCoTree performs a complementary unpacking operation through inverse dataflow scheduling. The near-bank PE obtains the data transmission from CoCoTree via periodic polling.

Figure 8(b) illustrates an example of this data packing process for a reduce operation. Co-Leaf 0 receives four 16-bit data from the PE: 0xA2A1, 0xB2B1, 0xC2C1, and 0xD2D1 (Step 1). The lower bytes of each word are grouped into a 4-byte packet to match the Co-Node internal bus width, forming 0xD1C1B1A1, and the upper bytes are also combined into another packet, 0xD2C2B2A2 (Step 2). This layout allows the Co-Node to perform element-wise operations efficiently at aligned byte granularity.

B. Co-Node Microarchitecture Design

Co-Node serves as the fundamental building block of CoCoTree, which supports configuring and directing data flows to designated destinations. As shown in Figure 8, each Co-Node integrates a configurable router, a set of functional units (FUs), and control logic to enable flexible inter-PE communication and support diverse collective operations. The details are as follows:

Routing and Control Logic: The *routing controller* (4) governs the forwarding behavior for both upward and downward data transfers. As illustrated in Figure 9, by coordinating the data selector (6) and handshake controller (1), it supports three downward (a–c) and four upward (d–g) transfer modes, enabling a wide range of communication patterns.

The *FU controller* (5) manages FUs (7) to perform computational operations, including addition, bitwise AND/OR/XOR and unsigned min/max. These computational resources provide hardware acceleration for collective operations. During the

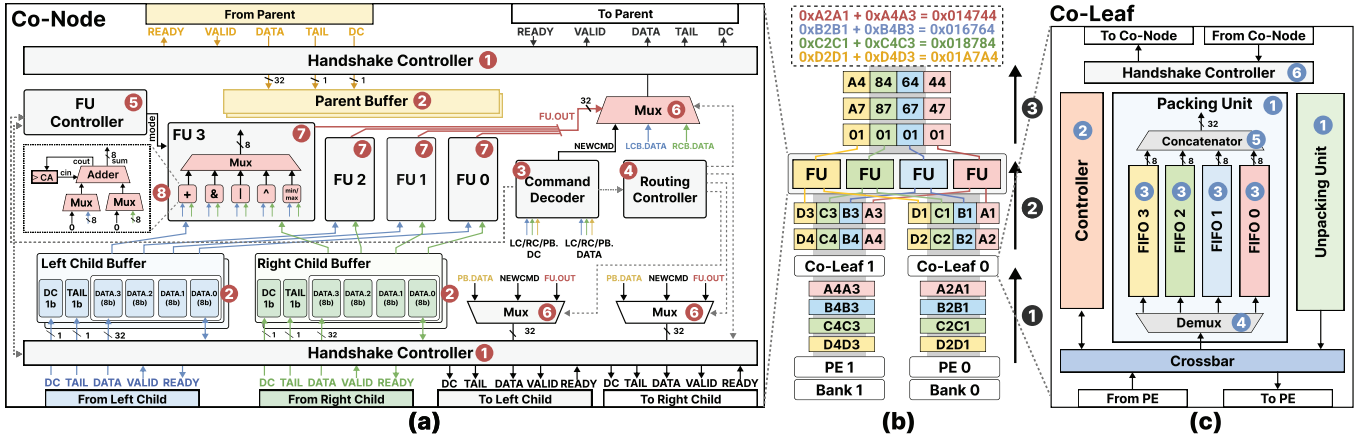


Fig. 8. (a) Detailed microarchitecture of Co-Node and (b) an example of data packing process for a reduce operation and (c) detailed microarchitecture of Co-Leaf.

configuration phase, each PE sends a command packet to configure the routing paths and FU function within the Co-Node. As shown in Figure 8(a), incoming packets are first processed by the handshake controller (1) and buffered accordingly (2). Packets from the left and right child nodes (or Co-Leaves) are placed into the corresponding child buffers, while packets from the parent Co-Node are stored in a dedicated parent buffer (PB). When the DC field in a packet is set to 0, it is identified as a configuration packet. The command decoder (3) parses its command field and configures the routing and FU controllers by updating internal registers.

Functional Unit: The Functional Units (FUs) (7) in the Co-Node support arbitrary byte-width integer and bitwise operations. Moreover, the FUs enable flexible integer reduction over various data types, such as 8/16/32/64 bits, and feature automatic overflow prevention via width adjustment. While this design supports integer and bitwise operations, floating-point computation can similarly be supported by incorporating an FPU module into the FU. The following discussion primarily uses integer operations as the main example.

During computation, each FU performs byte-granular processing on corresponding positions from the left and right child data packets. As depicted in Figure 8(a), the i th byte from each stream is fed to the dedicated functional unit FU_i , which performs operations such as sum or bit-wise operation. The result is written to the i th byte of the packet forwarded to the parent node. The FUs operate in a streaming manner. As each multi-byte data is split into a stream of byte-wise packets, their consistency must be maintained during FU processing. The design of FU incorporates consistency-aware flow control. For example, the adder (8) includes a carry accumulator (CA) to propagate carry bits between adjacent bytes. At the end of a stream, a non-zero CA triggers a temporary stall, outputs an overflow byte, and resets the carry register before resuming normal operation. A similar mechanism is used in min/max, which utilizes comparison registers to track the source of the smaller/larger value. As illustrated in Figure 8(b), four pairs of 16-bit values are sent to a Co-Node configured for reduce

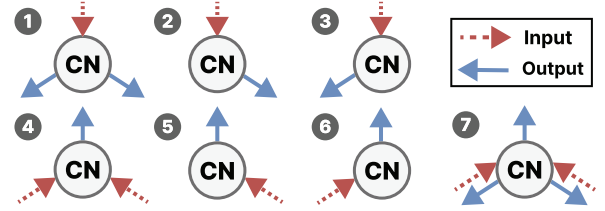


Fig. 9. Routing modes supported by Co-Nodes.

(Step 2), resulting in the data expanded from 16 bits to 24 bits width due to dynamic expansion of the Co-Node (Step 3).

C. Modularity Design

CoCoTree architecture provides dual modularity: functional modularity and structural modularity.

Functional Modularity. Inspired by the modular instruction set architecture of RISC-V, CoCoTree modularizes its supported functionalities, which correspond to the operations executed by its Functional Units (FUs). As detailed in Table I, these currently include basic functions (B), data transfer (T), bitwise reduction (RB), unsigned integer arithmetic reduction (RSU), floating-point arithmetic reduction (RSF), unsigned integer min/max reduction (RCU), and floating-point min/max reduction (RCF). Leveraging the flexible and modular architecture of CoCoTree, both the data width and the types of FUs can be customized based on the data types prevalent in the workloads. Such functional modularity allows the architecture to achieve an optimal cost-performance trade-off and offer flexibility under different budget constraints.

Structural Modularity. CoCoTree matches the intrinsic hierarchy of a DIMM. There exists a hierarchical tree structure within a DIMM: each DIMM consists of one or more ranks, each rank contains multiple chips, and each chip comprises several banks. CoCoTree similarly possesses a hierarchical tree structure. As illustrated in Figure 4, in each chip on the rank, N Co-Leaves ($N = 8$ in this work) and $N - 1$ Co-Nodes

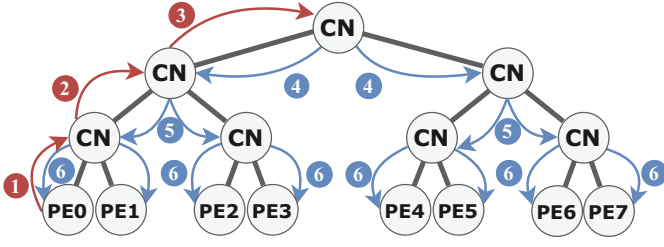


Fig. 10. Configuration and computation phases in CoCoTree communication. (CN: Co-Node)

TABLE I
CoCoTree SUPPORTED INSTRUCTIONS

Feat.	Name	Explanation
B	freeTree queryFeat	Release occupied Co-Nodes Query the features supported by Co-Nodes
T	transfer	P2P, Multicast, and Broadcast among PEs
RB	redAnd redOr redXor	Parallel bitwise AND reduction across PEs Parallel bitwise OR reduction across PEs Parallel bitwise XOR reduction across PEs
RSU	redSumU	Parallel unsigned integer sum reduction across PEs
RSF	redSumF	Parallel FP32 sum reduction across PEs
RCU	redMaxU redMinU	Parallel unsigned maximum reduction across PEs Parallel unsigned minimum reduction across PEs
RCF	redMaxF redMinF	Parallel FP32 maximum reduction across PEs Parallel FP32 minimum reduction across PEs

are connected directly to form a chip-level binary tree. In each rank, M chips ($M = 8$ in this work) are connected to a rank-level CoCoTree chip consisting of $M - 1$ Co-Nodes via a bi-directional SerDes link. Similarly, at the next level, a comparable binary tree structure can be established through appropriate connection serial links as listed in Table II. Such LEGO-like composability and self-similar hierarchical organization provide CoCoTree with significant scalability, supporting collaborative processing across an expandable array of near-bank PEs.

VI. CoCoTree COMMUNICATION MECHANISM

A. Communication Workflow

CoCoTree adopts a two-phase communication model and decouples control flow and data flow into a *configuration phase* and a *computation phase*. The separate configuration phase eliminates redundant transmission of metadata like destination addresses and operation types, increasing bandwidth efficiency during the execution of collective operations.

Communication in CoCoTree is packet-based, with two distinct packet types: *command packets* and *data packets*. Prior to data transmission, a command packet must first be issued to configure the routers and functional units within each Co-Node of the designated (sub)tree. Once all relevant Co-Nodes are configured, Co-Leafs initiate data transmission from the PEs into the CoCoTree interconnect, where packets are routed through the network based on the established configuration.

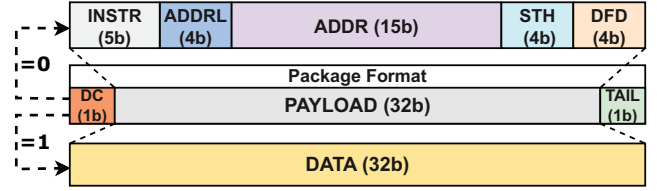


Fig. 11. 32-bit CoCoTree Packet Format.

TABLE II
INTERCONNECT LINK TYPE

Level	Link Type	Reach	Bandwidth
Rank-level	GRS [70] on PCB	80mm	25Gb/s/pin
DIMM-level	Ribbon Cable [28]	500mm	16Gb/s/pin

Configuration Phase. During this phase, host CPU selects one PE to send a command packet to specify the communication pattern, collective operation type, and the participating PEs. As illustrated in Figure 10, the command packet first traverses upward to the local root of the designated tree (1–2–3), and is then broadcast downward to all nodes within the tree (4–5–6). Each Co-Node along this path parses the command and configures its internal routing and computation units accordingly. The simultaneous arrival of the command packet at all PEs acts as a synchronization barrier, triggering each PE to commence data transmission.

Computation Phase. During the computation phase, involved PEs will send data packets through the Co-Leafs to the CoCoTree, where the data will transfer upward through the Co-Nodes. At each node, data transmission or processing is performed. This process continues until the final results reaches the local root node. Figure 10 shows the computation phase of the CoCoTree for reduce operation. As the data ascends through the tree, configured FUs in each Co-Node perform a computation on the data from its left and right children, sending the partial result to its parent node until it ultimately reaches the root node (Steps 1–2–3). The configured routers in the Co-Nodes will then send the final aggregate result down to the designated destination (Steps 4–5–6), completing the collective communication.

B. CoCoTree Packet and Protocol Design

CoCoTree adopts a lightweight, address-driven protocol that leverages the structural regularity of a perfect binary tree to achieve efficient routing and support the above communication mechanism. Figure 11 illustrates the 32-bit packet format used in CoCoTree. Each packet is categorized by the DC field, which distinguishes between data and command packets. Data packets carry payload as DATA and include a TAIL bit to indicate the end of a stream. Command packets specify operation modes or management instructions for CoCoTree, encoded in the INSTR field, as detailed in Table I. The ADDR field indicates the bit-length of the destination address encoded in the ADDR field. ADDR identifies the target Co-Leaf node or subtree. The STH (Sub-Tree-Height) field defines the

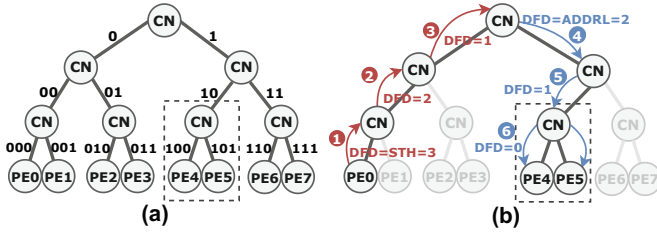


Fig. 12. (a) Explanation of the ADDR field in the protocol and (b) routing decision in CoCoTree. (CN: Co-Node)

height of the target subtree, with 2^{STH} PEs participating in the operation. By adjusting STH, the system can flexibly scale the degree of parallelism involved in a collective operation, allowing distinct PE groups to communicate independently and in parallel within their respective subtrees.

To facilitate destination resolution and forwarding, each packet also contains a DFD (Distance-From-Destination) field. DFD denotes the number of hops remaining to reach the target node. When a command packet is issued by a PE, the DFD is initialized to STH; if the command originates from the local root, DFD is modified to ADDRL. As the packet traverses each Co-Node, the DFD decrements by one. A DFD value of 1 signals arrival at the destination node, while a DFD value of 0 in the packet indicates that the packet has reached the designated subtree.

An example is shown in Figure 12(a). During the configuration phase, the ADDR field undergoes a left rotation at each Co-Node before being forwarded to child nodes. When $\text{DFD} \neq 0$, routing decisions are made based on the least significant bit (LSB) of the ADDR: a 0 directs the packet to the left child, and a 1 to the right child. Once the packet enters the target subtree ($\text{DFD} = 0$), it is broadcasted to both child nodes simultaneously. Figure 12(b) illustrates this dynamic using a command packet with ADDR = b10 and ADDRL = 2, targeting the subtree that includes PE 4 and PE 5, and demonstrates how DFD evolves during routing.

C. Stream Control

In order to support the flexible regulation of data packets flow among Co-Nodes, CoCoTree incorporates a handshake-based stream control inspired by the valid/ready handshake protocol in AXI4 [3]. In CoCoTree, each link between nodes is equipped with `valid` and `ready` handshake signals. The `valid` signal is asserted by the source node to indicate that a data packet is available on the channel, while the `ready` signal is driven by the destination node to indicate its ability to accept incoming data. A data transfer only occurs when both signals are asserted in the same cycle. The destination can temporarily stall the stream by deasserting `ready`, while the source can invalidate a transfer opportunity by lowering `valid`. Within the CoCoTree hierarchy, this fine-grained control allows each Co-Node to stall or resume packet flow based on local buffer availability and pipeline status, preventing data loss and avoiding unnecessary stalling across the network.

TABLE III
SYSTEM CONFIGURATION.

Configuration of Host Server	
CPU Model	2×Intel Xeon Silver 4216
CPU Clock Frequency	2.2GHz
Number of Cores	32
Memory Capacity	256GB
Configuration of UPMEM PIM	
DIMM Type	20×UPMEM BC021B
DPU Clock Frequency	350MHz
Total Number of PEs	2530
Memory Capacity	160GB
Memory Specification	DDR4-2400
Parameter of Tools	
UPMEM SDK	upmem-2023.2.0-Linux-x86_64
Compiler	G++ 12.3.0

Additionally, a `tail` signal is used signify the end of a data packet stream. The `tail` flag is asserted only when the final packet of a stream is transmitted, allowing the destination node to detect the end-of-transmission and trigger appropriate state transitions or processing. In hardware, there are Handshake Controllers (1, 6) to handle the stream control as shown in Figure 8(a)(c).

VII. EVALUATION

A. Evaluation Methodology

Experiment Setup. We implement the CoCoTree architecture in Chisel [8], a hardware construction language designed for agile hardware development, and then compile to Verilog. To evaluate the design, we integrate the RTL implementation with Verilator for cycle-accurate simulation, and we also develop an cycle-accurate simulator in C++ using the DPI-C interface for PIM workload simulation. This simulator is able to model the collective communication operations in CoCoTree and enables detailed performance analysis for PIM workload. Our PIM system configuration is based on a real-world commodity UPMEM DIMM PIM server. The system configuration is summarized in Table III.

Baselines. We evaluate both the performance of supported collective communication operations and the end-to-end performance of various PIM applications [37]. We compare CoCoTree against three representative DIMM PIM baselines: (1) Basic DIMM PIM (UPMEM), where collective communication operations are handled by the host CPU; (2) DIMM-Link, which introduces direct point-to-point links between DIMMs. However, DIMM-link is not optimized for inter-PE communication; and (3) PIMnet, which supports intra-channel collective operations via a multi-tier network. For PE number over 256 (a channel in UPMEM), PIMnet still needs host CPU forwarding. For fair comparison, the evaluation on basic DIMM PIM is directly executed on our UPMEM server configured as shown in Table III. The experimental results for DIMM-Link and PIMnet are obtained through simulation using reported link bandwidth from their papers.

Benchmarks. Table IV lists the benchmarks used in evaluation. Following previous research [35], [37], [67], we evaluate CoCoTree using a series of memory-intensive workloads: Breadth-First Search (BFS), Histogram (HST), Connected

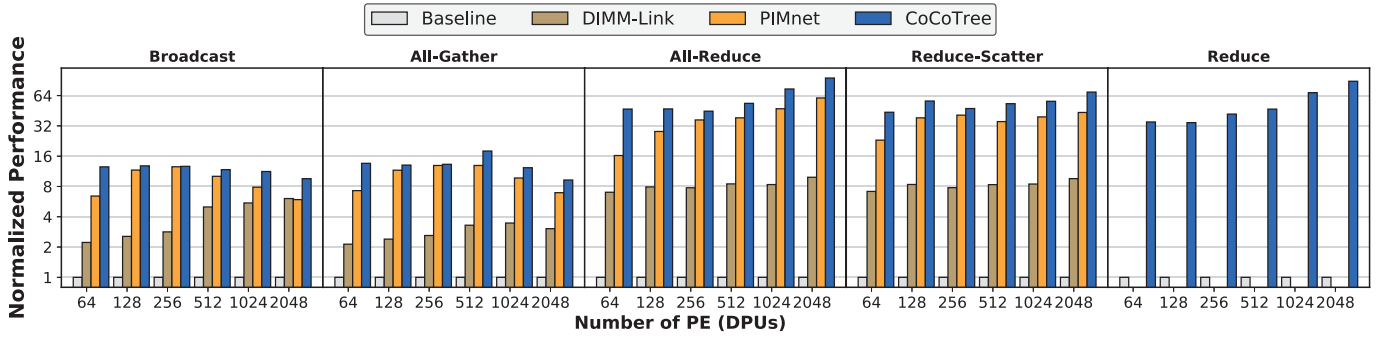


Fig. 13. Performance comparison of different collective communication operations under varying PE counts across PIM communication architectures, results are normalized to UPMEM PIM baseline

TABLE IV
BENCHMARKING APPLICATIONS

PIM Workload	Abbr.	Comm.	Input/Config
Breadth-First Search	BFS	All-Reduce	rMat [12]
Histogram	HST	All-Reduce	1536 × 1024
Reduction	RED	Reduce	6.3M elems
Multi-layer Perceptron	MLP	Reduce-Scatter	256 × 256
Matrix-vector multiplication	GEMV	Reduce-Scatter	1024 × 64
Sparse matrix-vector multiplication	SpMV	Reduce-Scatter	rtn [35]
Connected Component	CC	All-Reduce	rMat [12]
Embedding Lookup	EMB	Reduce-Scatter	RM2 [63]

Component (CC), Multi-layer Perceptron (MLP), General Matrix-vector multiplication (GEMV), Reduction (RED) from [37], Sparse Matrix-vector Multiplication (SpMV) [35], and Embedding Lookup (EMB) [67]. These workloads are selected since they are widely used to evaluate previous PIM architectures [37], [76], [86] and they involve heavy inter-PE communication, making them highly sensitive to the efficiency of collective communication operations. Collective communication in these applications has significant influence on overall execution performance. All programs on the host CPU are conducted with OpenMP enabled.

B. Experimental Results Analysis

1) Collective Communication Performance Comparison:

We compare the throughput performance of CoCoTree against baseline PIM systems under various collective communication operations. Throughput is defined as the size of the larger communication operand (e.g., input side for *Reduce*) divided by the execution time. All experiments adopt the weak scaling setup, where each PE processes a fixed 8KB data. The evaluated collective operations include *Broadcast*, *All-Gather*, *Reduce*, *Reduce-Scatter*, and *All-Reduce*. Note that DIMM-Link and PIMnet architectures do not support native *Reduce* operations, and are therefore excluded from the *Reduce* throughput comparison.

As shown in Figure 14, we first evaluate the baseline UPMEM system. The throughput of *Broadcast* and *All-Gather* slightly increases with the number of PEs, but remains low even at 2048 PEs. Other evaluated collective operations show even lower throughput for all scales. These results indicate that inter-PE communication in current DIMM PIM has become a

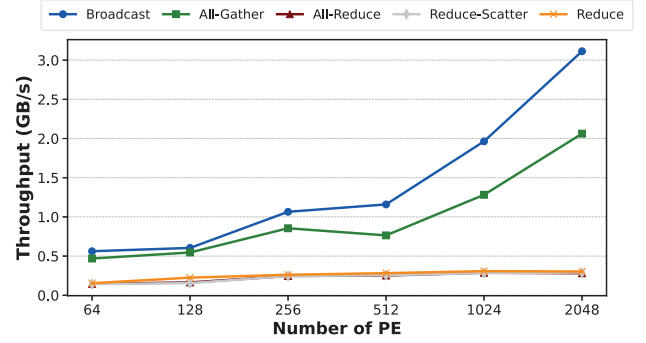


Fig. 14. Collective communication throughput across varying PE counts of baseline UPMEM DIMM PIM

major performance bottleneck, highlighting the need for more efficient PIM communication architectures.

We then evaluate throughput under different PIM communication architectures, with results summarized in Figure 13. CoCoTree significantly improves the throughput across all collective operations. For *All-Reduce*, CoCoTree achieves up to 95.6× speedup and an average of 60.4× over the UPMEM baseline. For *Reduce* and *Reduce-Scatter* operations, it gains average speedup of 54.5× and 54.4×, respectively. Compared to DIMM-Link, CoCoTree offers an average 5.9× improvement, owing to its tree topology and in-network computation support. Against PIMnet, CoCoTree achieves comparable gains in *Broadcast* and *All-Gather* (1.4× on average), and larger improvements in *Reduce-Scatter* (1.5×) and *All-Reduce* (1.7×), where host intervention for inter-channel traffic in PIMnet limits scalability. These results demonstrate that CoCoTree effectively enables high-throughput collective communication in DIMM PIM systems by offloading communication from the host CPU. Its benefits are most significant in multi-source data fusion workloads such as *All-Reduce* and *Reduce-Scatter*.

2) *Scalability of Collective Communication:* To evaluate the scalability of CoCoTree in DIMM PIM systems, we scale the number of PEs from 64 to 2048 and measure the performance across selected collective communication operations. The scalability for baseline UPMEM PIM system can be observed in Figure 14. For *Broadcast* and *All-Gather*, the

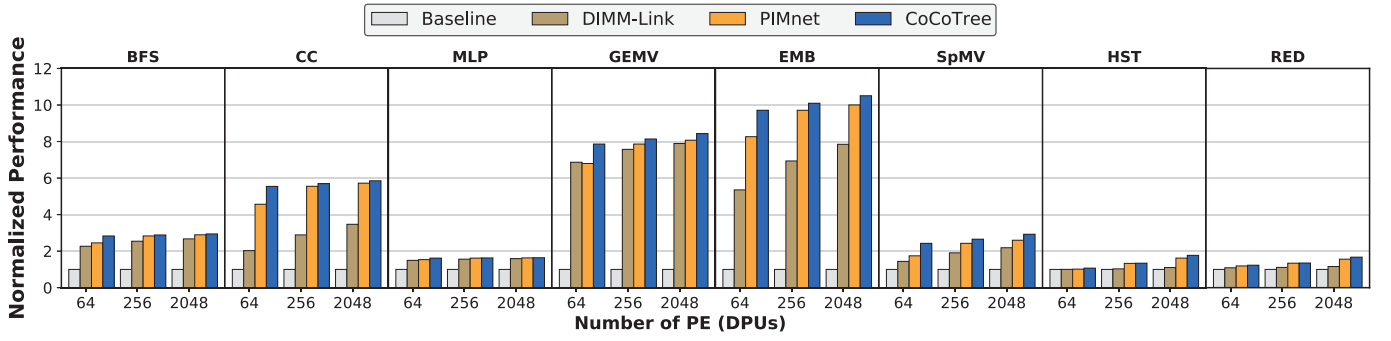


Fig. 15. Performance comparison between CoCoTree and baselines across PIM workloads, results are normalized to the UPMEM PIM baselines

throughput increases slowly with the number of PE, as these operations can leverage the broadcast and parallel transfer functions provided by the UPMEM SDK [24]. For other operations like *All-Reduce*, the lack of inter-PE communication support results in almost no throughput improvement regardless of scale, confirming that host CPU-forwarding limits the scalability.

Figure 13 illustrates the scalability of CoCoTree and baselines. CoCoTree maintains high speedups over the baseline across all scales, and the performance gap widens as the number of PEs increases. We attribute this to the computation-capable tree network of CoCoTree, which provides efficient and scalable communication. The hierarchical structure minimizes scaling cost and enables in-network computation via Co-Nodes during data transmission. DIMM-Link partially mitigates CPU bottlenecks by introducing dedicated inter-DIMM links and offloading communication tasks to NMP cores on buffer chips. While this improves scalability over the baseline, the increasing network scale causes performance to stagnate beyond 512 PEs. PIMnet further improves intra-channel communication by supporting direct data exchange among PIM banks. PIMnet still requires the host CPU for inter-channel coordination. This reliance limits the scalability when the number of PEs grows beyond a single channel.

3) *PIM Workloads Performance Analysis*: To evaluate the performance benefits of CoCoTree, we compare CoCoTree against the baselines on selected PIM workloads listed in Table IV. The results are shown in Figure 15. Basic DIMM PIM baseline is implemented on the UPMEM platform. For DIMM-Link, PIMnet, and CoCoTree, we replace the host-forwarding communication in UPMEM PIM with the collective communication supported by each PIM communication architecture.

Compared to the UPMEM baseline, CoCoTree significantly improves performance across all evaluated PIM workloads. For EMB and GEMV workloads, the *Reduce-Scatter* dominates the total execution time in the baseline system. By offloading the *Reduce-Scatter* to the hierarchical tree network, CoCoTree achieves up to $10.5\times$ and $8.4\times$ end-to-end speedup. Graph workloads such as BFS and CC rely on *All-Reduce* to synchronize graph node information across multiple PIM PEs. CoCoTree achieves up to $2.9\times$ and $5.7\times$ speedup,

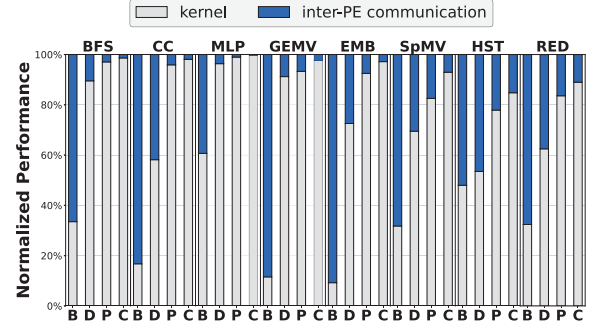


Fig. 16. Execution time breakdown across benchmarks. (B: UPMEM host communication baseline, D: DIMM-Link, P: PIMnet, C: CoCoTree)

respectively. A greater performance improvement is observed in CC because it involves more communication than BFS. We achieve a speedup of up to $2.9\times$ for SpMV by accelerating the *Reduce-Scatter* communication, which bypasses costly inter-PE communication through the host. MLP, HST, and RED all achieve a relatively modest end-to-end speedup of up to $1.6\times$, $1.8\times$, and $1.6\times$, respectively. This is because the effectiveness of communication optimizations is limited in these workloads, whether due to a high proportion of compute time (MLP) or fewer collective communication requirements (HST and RED).

Compared to DIMM-Link and PIMnet, CoCoTree delivers consistently comparable or higher performance across all workloads due to its more efficient and scalable collective communication support. For workloads at 2048 PEs, CoCoTree achieves up to $1.7\times$ and on average $1.3\times$ speedup against DIMM-Link. Compared with PIMnet, CoCoTree achieves $1.1\times$ speedup on average. Furthermore, Figure 15 also illustrates the scalability of these communication architectures on all evaluated applications. Experimental results show that for these real-world workloads, CoCoTree maintains scalability, and the performance improvement of CoCoTree remains stable and in several cases even slightly increases as the number of PEs increases from 64 to 2048.

4) *Time Breakdown Analysis*: To reveal how CoCoTree alleviates the communication bottleneck in DIMM PIM systems, we perform a detailed breakdown of the execution time across various workloads, quantifying the proportion of time spent on inter-PE communication. Figure 16 illustrates

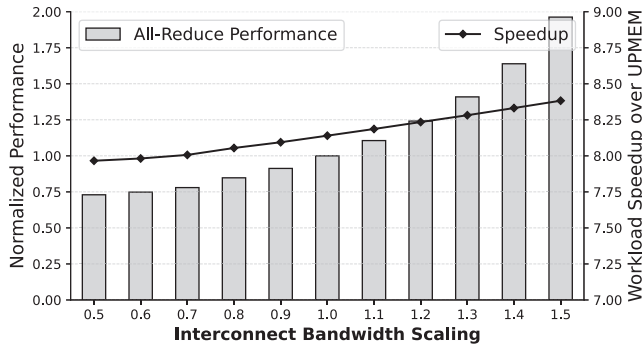


Fig. 17. All-reduce performance and application-level acceleration across different interconnect bandwidth configurations.

the time breakdown for the real-world PIM workloads under different PIM communication architecture. In the baseline system, all collective communications are forwarded by the host CPU, making inter-PE communication the dominant source of overhead. For communication-intensive applications such as *CC*, inter-PE overhead accounts for up to 82.0% of the PIM execution time.

CoCoTree offloads both communication and computation onto a hierarchical hardware interconnect, significantly reducing the inter-PE communication fraction. Across all evaluated workloads, the IPC overhead is reduced to an average of 5.3%, and even falls below 0.5% in some certain benchmarks. This indicates communication is no longer the primary performance bottleneck in the CoCoTree system. Furthermore, since CoCoTree performs *Reduce*, *Scatter*, and other collective operations directly within the network, it eliminates the overhead of additional WRAM-MRAM data copy and synchronization in PIMnet. Consequently, CoCoTree transforms inter-PE communication from a dominant and system-wide bottleneck into a minor overhead through architecture and protocol co-design.

5) *Robustness Analysis Under Bandwidth Variation*: To evaluate the robustness of CoCoTree under different interconnect link bandwidth configurations, we analyze the communication and application performance when varying link bandwidth. We measure the execution time of the *All-Reduce* and the end-to-end performance for GEMV application under different link widths. As shown in Figure 17, even if the interconnect bandwidth is reduced to 50%, CoCoTree can still achieves up to $8.0\times$ speedup over baseline UPMEM PIM. This indicates that its efficiency does not rely on aggressive bandwidth provisioning, but instead stems from its concurrent and structurally optimized hierarchical tree design. Moreover, when bandwidth increases, CoCoTree continues to scale and delivers improved performance. It gains a $2.0\times$ performance improvement in *All-Reduce* when bandwidth increases 50%. This demonstrates that the CoCoTree can further exploit additional communication resources.

6) *Ablation Study*: We conduct an ablation study to quantify the performance contribution of the components of CoCoTree: (N) Tree network, (C) In-network Computation, and (P) Pipelining, on the (B) baseline DIMM PIM system. Figure

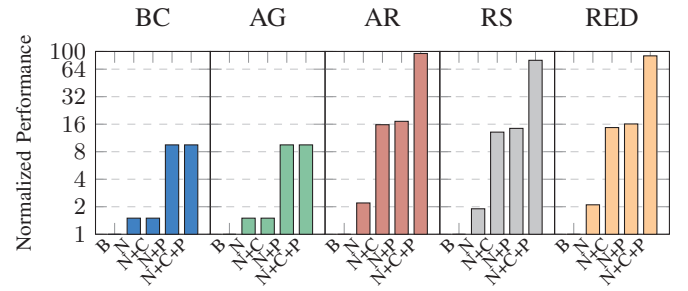


Fig. 18. Ablation study of CoCoTree components. All results are normalized to DIMM PIM Baseline. (B:Baseline, N:Tree Network, C:Compute, P:Pipeline, BC:Broadcast, AG:All-Gather, AR:All-Reduce, RS:Reduce-Scatter, RED:Reduce)

18 reports the normalized performance for five collective primitives at 2048 PEs.

We first examine the effect of the tree network alone. (N) The tree network (FUs disabled) provides a high-bandwidth path bypassing the host CPU, providing $1.5\times$ speedup for *Broadcast* and *All-Gather* and $1.9\times$ to $2.2\times$ speedup for the rest operations. Adding in-network computation (N+C) significantly improves computation-heavy collective operations like *All-Reduce*, *Reduce-Scatter* and *Reduce*, achieving $14.5\times$ speedup on average. Enabling in-network computation allows the system to offload operations into the tree and halve the traffic at each level. But FUs offer little benefit to computation-free collectives like *Broadcast* and *All-Gather*. Then, We find that Pipelining further improves performance by overlapping rounds, hiding network latency. With only the tree network and pipelining enabled (N+P), performance improves from $9.5\times$ to $17.2\times$ for evaluated collective communication operations. The full CoCoTree (N+C+P) consistently delivers the highest performance and the results have been shown in the collective communication performance comparison subsection. Overall, the ablation confirms that CoCoTree provides its best performance when the tree network, in-network computation, and pipelining are jointly employed.

7) *Hardware Overhead Analysis*: To evaluate the area and power characteristics of the CoCoTree architecture, we employ an open-source EDA flow utilizing Yosys [81] for RTL synthesis, iEDA [58] for physical implementation and place-and-route, both based on a 45nm technology (NanGate45). Previous study [24] has shown that DRAM process incurs larger area overhead than ASIC due to the reduced number of metal layers in DRAM technology. Therefore, we employ a scaling factor of 10 to evaluate the area overhead for DRAM process [24]. Our analysis focuses on the area and power overhead of Co-Leaf and Co-Node units. The results show that compared to the current commodity DIMM PIM bank [23], each Co-Leaf unit only brings 0.5% area overhead and 0.4% power consumption overhead.

Table V provides a breakdown of the area and power overhead contributed by various features. Each Co-Node unit supporting transfer, bitwise, and integer reduction (T, RB, RSU, RCU features) incurs an area overhead of 0.030mm^2

and a power overhead of 1.36mW. Extending support to include floating-point operations (i.e. adding RSF, RCF features) increases the overhead to 0.076mm² and 3.78mW. For an 8-PE tree interconnect on a single chip, the aggregated overhead for all Co-Nodes is 0.20mm² and 9.76mW. We consider this hardware cost negligible, especially given the significant improvements in collective communication performance, and it does not compromise manufacturability. Notably, when Co-Nodes are deployed hierarchically within buffer chips or controllers, their overhead is marginal compared to the resources of these host components [45].

TABLE V
AREA/POWER ANALYSIS OF CO-NODE ACROSS DIFFERENT FEATURE

T	RB	RSU	RCU	RSF	RCF	Area/mm ²	Power/mW
✓						0.018	0.70
✓	✓					0.026	1.12
✓	✓	✓				0.030	1.36
✓	✓	✓	✓			0.071	3.52
✓	✓	✓	✓	✓		0.076	3.78
✓				✓	✓	0.065	3.05

VIII. RELATED WORK

DIMM PIM Architecture. In recent years, research has explored accelerating diverse workloads using DIMM-based PIM architectures. DIMM PIM provides the high aggregated memory bandwidth and parallelism for workloads. Previous studies have demonstrated the acceleration of machine learning [23], [39], [57], graph pattern matching [11], vector search [60] and sequencing analysis [15] on commercial UPMEM PIM platforms [24]. ReCross [61] leverages cross-level DIMM NMP architecture to speed up personalized recommendation. MEDAL [43] proposes a specialized DIMM-based NMP accelerator for DNA sequence alignment. [75] explores the DIMM PIM architecture for accelerating time series analysis. Other researches such as DIMMining [21] and Hermes [62] target graph mining and large language model inference acceleration, respectively.

Inter-PE Communication for PIM. Most existing DIMM PIM systems rely on host CPU forwarding for inter-PE communication, which limits the scalability. To mitigate this, recent studies explore direct communication mechanisms. SimplePIM [14] offers collective APIs at the software level but still requires the host CPU. DIMM-Link [86] proposes physical links between DIMMs and enable inter-DIMM communication. NDPBridge [79] provides hardware bridges across the DRAM hierarchy to accelerate communication between banks. PIMnet [76] proposes a hierarchical interconnect with integrated control. Our CoCoTree introduces a structured tree-based topology with in-network computation enabled by functional units embedded at routing nodes, offering efficient and scalable collective communication for DIMM PIM systems.

IX. DISCUSSION

Root Bottleneck: CoCoTree does not suffer from a severe root bottleneck. In this design, the communication load is

kept balanced across all Co-Nodes, including the local root. By coupling in-network computation with transport, it halves traffic volume at each reduction level, preventing the local root from becoming a dominant bottleneck. Furthermore, CoCoTree supports configurable subtrees, enabling multiple concurrent roots and parallel collectives. However, CoCoTree is less optimal for highly irregular or permutation-heavy traffic (e.g., all-to-all), where tree routing is suboptimal. A viable approach is to selectively employ CoCoTree for collectives and utilize existing paths, such as host-forwarding, for such permutation-heavy operations.

Multi-tenancy: While typical PIM systems often target a single workload, CoCoTree is designed to efficiently support variable-scale multi-tenancy. Thanks to its flexible protocol design, CoCoTree allows multiple disjoint collectives to run concurrently. The reconfiguration process is lightweight, merely involving reissuing commands to the affected subtree rather than altering the entire tree.

Memory Layout: Memory layout also impacts scheduling and communication [86]. But current DIMM-based DIMM platform lacks mechanisms that efficiently support general and fine-grained memory layout and scheduling. So smart memory layout designs [1], [29], [35] cannot be directly implemented on current DIMM PIM hardware. Also, many workloads with global synchronization or reductions, such as graph algorithms [37], [76], [79], [84] and machine learning [39], [80], inherently require inter-PE communication that optimized layouts cannot eliminate. We mainly focus on the PIM communication architecture design in this work, but combining CoCoTree with a efficient memory layout design can bring better performance for the system.

X. CONCLUSION

In this work, we propose CoCoTree, a computation-capable architecture for collective communication in scalable DIMM-based PIM systems. By leveraging a hierarchical binary-tree interconnect and in-network computation, CoCoTree enables highly efficient and direct inter-PE collective communication without the host CPU forwarding. We propose the system organization, the microarchitecture design of key components Co-Node and Co-Leaf, and the communication mechanism for CoCoTree. Our experiments show that compared to the CPU-forwarding UPMEM baseline, CoCoTree achieves up to 95.6× speedup on collective communication operations and 10.5× performance improvement in PIM workloads, and up to 1.7× improvement on collective communication operations against the state-of-the-art PIM communication network.

ACKNOWLEDGEMENTS

We sincerely thank the anonymous reviewers of HPCA for their insightful comments and suggestions. This work was partially supported by the National Natural Science Foundation of China under Grant No. 62488101, No. 62495104 and No. 62202454. Xueqi Li is the corresponding author.

REFERENCES

- [1] J. Ahn, S. Hong, S. Yoo, O. Mutlu, and K. Choi, "A scalable processing-in-memory accelerator for parallel graph processing," in *2015 ACM/IEEE 42nd Annual International Symposium on Computer Architecture (ISCA)*, 2015, pp. 105–117.
- [2] J. Ahn, S. Yoo, O. Mutlu, and K. Choi, "PIM-enabled instructions: A low-overhead, locality-aware processing-in-memory architecture," in *2015 ACM/IEEE 42nd Annual International Symposium on Computer Architecture (ISCA)*, 2015, pp. 336–348.
- [3] AMD, "AMBA® AXI4 Interface Protocol," <https://www.amd.com/en/products/adaptive-socs-and-fpgas/intellectual-property/axi.html>, 2025, [Accessed 01-08-2025].
- [4] S. Angizi, N. A. Fahmi, W. Zhang, and D. Fan, "PIM-Assembler: A processing-in-memory platform for genome assembly," in *2020 57th ACM/IEEE Design Automation Conference (DAC)*, 2020, pp. 1–6.
- [5] S. Angizi and D. Fan, "GraphiDe: A graph processing accelerator leveraging In-DRAM-Computing," in *Proceedings of the 2019 Great Lakes Symposium on VLSI*, 2019, p. 45–50.
- [6] S. Angizi, J. Sun, W. Zhang, and D. Fan, "PIM-Aligner: A Processing-in-MRAM platform for biological sequence alignment," in *2020 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2020, pp. 1265–1270.
- [7] H. Asghari-Moghaddam, Y. H. Son, J. H. Ahn, and N. S. Kim, "Chameleon: Versatile and practical near-dram acceleration architecture for large memory systems," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016, pp. 1–13.
- [8] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avizienis, J. Wawrzyniek, and K. Asanović, "Chisel: constructing hardware in a scala embedded language," in *Proceedings of the 49th Annual Design Automation Conference*, 2012, p. 1216–1225.
- [9] P. Bhat, C. Raghavendra, and V. Prasanna, "Efficient collective communication in distributed heterogeneous systems," in *Proceedings. 19th IEEE International Conference on Distributed Computing Systems (Cat. No.99CB37003)*, 1999, pp. 15–24.
- [10] A. Boroumand, S. Ghose, B. Akin, R. Narayanaswami, G. F. Oliveira, X. Ma, E. Shiu, and O. Mutlu, "Google neural network models for edge devices: Analyzing and mitigating machine learning inference bottlenecks," in *2021 30th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2021, pp. 159–172.
- [11] S. Cai, B. Tian, H. Zhang, and M. Gao, "PimPam: Efficient graph pattern matching on real processing-in-memory hardware," *Proc. ACM Manag. Data*, vol. 2, no. 3, 2024.
- [12] D. Chakrabarti, Y. Zhan, and C. Faloutsos, "R-MAT: A recursive model for graph mining," in *Proceedings of the Fourth SIAM International Conference on Data Mining, Lake Buena Vista, Florida, USA, April 22-24, 2004*, 2004, pp. 442–446.
- [13] E. Chan, M. Heimlich, A. Purkayastha, and R. van de Geijn, "Collective communication: theory, practice, and experience: Research articles," *Concurr. Comput.: Pract. Exper.*, vol. 19, no. 13, pp. 1749–1783, 2007.
- [14] J. Chen, J. Gómez-Luna, I. El Hajj, Y. Guo, and O. Mutlu, "SimplePIM: A Software Framework for Productive and Efficient Processing-in-Memory," in *2023 32nd International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2023, pp. 99–111.
- [15] L.-C. Chen, C.-C. Ho, and Y.-H. Chang, "UpPipe: A novel pipeline management on in-memory processors for RNA-seq quantification," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*, 2023, pp. 1–6.
- [16] S. Chen, H. Tan, A. C. Zhou, Y. Li, and P. Balaji, "Updlrm: Accelerating personalized recommendation using real-world pim architecture," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024.
- [17] F. Cheng, T. Zhang, J. Zhang, J. H.-C. Ku, Y. Wang, X. Yang, Hai, Li, and Y. Chen, "Autorac: Automated processing-in-memory accelerator design for recommender systems," 2025.
- [18] M. Cho, U. Finkler, and D. Kung, "Blueconnect: Novel hierarchical all-reduce on multi-tired network for deep learning," in *Proceedings of the 2nd SysML Conference*, 2019.
- [19] J. Cong, Z. Fang, M. Gill, F. Javadi, and G. Reinman, "AIM: Accelerating computational genomics through scalable and noninvasive accelerator-interposed memory," in *Proceedings of the International Symposium on Memory Systems*, 2017, pp. 3–14.
- [20] M. Cowan, S. Maleki, M. Musuvathi, O. Saarikivi, and Y. Xiong, "MSC-CLang: Microsoft collective communication language," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2023, p. 502–514.
- [21] G. Dai, Z. Zhu, T. Fu, C. Wei, B. Wang, X. Li, Y. Xie, H. Yang, and Y. Wang, "Dimmining: pruning-efficient and parallel graph mining on near-memory-computing," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, p. 130–145.
- [22] A. Dakkak, C. Li, J. Xiong, I. Gelado, and W.-m. Hwu, "Accelerating reduction and scan using tensor core units," in *Proceedings of the ACM International Conference on Supercomputing*, 2019, p. 46–57.
- [23] P. Das, P. R. Sutradhar, M. Indovina, S. M. P. Dinakarrao, and A. Ganguly, "Implementation and evaluation of deep neural networks in commercially available processing in memory hardware," in *2022 IEEE 35th International System-on-Chip Conference (SOCC)*, 2022, pp. 1–6.
- [24] F. Devaux, "The true processing in memory accelerator," in *2019 IEEE Hot Chips 31 Symposium (HCS)*, 2019, pp. 1–24.
- [25] S. Diab, A. Nassereldine, M. Alser, J. Gómez Luna, O. Mutlu, and I. El Hajj, "A framework for high-throughput sequence alignment using real processing-in-memory systems," *Bioinformatics*, vol. 39, no. 5, p. btad155, 2023.
- [26] S. Diab, A. Nassereldine, M. Alser, J. G. Luna, O. Mutlu, and I. E. Hajj, "High-throughput pairwise alignment with the wavefront algorithm using processing-in-memory," in *2022 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2022, pp. 163–163.
- [27] A. Farmahini-Farahani, J. H. Ahn, K. Morrow, and N. S. Kim, "NDA: Near-DRAM acceleration architecture leveraging commodity DRAM devices and standard memory modules," in *2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*, 2015, pp. 283–295.
- [28] M. Gao, G. Ayers, and C. Kozyrakis, "Practical near-data processing for in-memory analytics frameworks," in *2015 International Conference on Parallel Architecture and Compilation (PACT)*, 2015, pp. 113–124.
- [29] M. Gao, J. Pu, X. Yang, M. Horowitz, and C. Kozyrakis, "Tetris: Scalable and efficient neural network acceleration with 3d memory," in *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*, 2017, p. 751–764.
- [30] M. Gao, M. Coletti, R. B. Davidson, R. Prout, S. Abraham, B. Hernández, and A. Sedova, "Proteome-scale deployment of protein structure prediction workflows on the summit supercomputer," in *2022 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2022, pp. 206–215.
- [31] S. M. Ghazimirsaeed, Q. Anthony, A. Shafi, H. Subramoni, and D. K. D. Panda, "Accelerating GPU-based machine learning in python using MPI library: A case study with MVAPICH2-GDR," in *2020 IEEE/ACM Workshop on Machine Learning in High Performance Computing Environments (MLHPC) and Workshop on Artificial Intelligence and Machine Learning for Scientific Applications (A4S)*, 2020, pp. 1–12.
- [32] A. Gholami, Z. Yao, S. Kim, C. Hooper, M. W. Mahoney, and K. Keutzer, "AI and memory wall," *IEEE Micro*, vol. 44, no. 3, pp. 33–39, 2024.
- [33] S. Ghosh, M. Halappanavar, A. Kalyanaraman, A. Khan, and A. H. Gebremedhin, "Exploring MPI communication models for graph applications using graph matching as a case study," in *2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2019, pp. 761–770.
- [34] S. Ghosh, N. R. Tallent, and M. Halappanavar, "Characterizing performance of graph neighborhood communication patterns," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 4, pp. 915–928, 2022.
- [35] C. Giannoula, I. Fernandez, J. Gómez-Luna, N. Koziris, G. Goumas, and O. Mutlu, "SparseP: Towards efficient sparse matrix vector multiplication on real processing-in-memory architectures," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 6, no. 1, 2022.
- [36] M. Gokhale, S. Lloyd, and C. Hajas, "Near memory data structure rearrangement," in *Proceedings of the 2015 International Symposium on Memory Systems*, 2015, p. 283–290.
- [37] J. Gómez-Luna, I. E. Hajj, I. Fernandez, C. Giannoula, G. F. Oliveira, and O. Mutlu, "Benchmarking a New Paradigm: Experimental Analysis and Characterization of a Real Processing-in-Memory System," *IEEE Access*, vol. 10, pp. 52 565–52 608, 2022.

- [38] Y. Gu, A. Khadem, S. Umesh, N. Liang, X. Servot, O. Mutlu, R. Iyer, and R. Das, "PIM Is All You Need: A CXL-Enabled GPU-Free System for Large Language Model Inference," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025, pp. 862–881, arXiv:2502.07578 [cs].
- [39] J. Gómez-Luna, Y. Guo, S. Brocard, J. Legriel, R. Cimadomo, G. F. Oliveira, G. Singh, and O. Mutlu, "Machine Learning Training on a Real Processing-in-Memory System," in *ISVLSI*, 2022.
- [40] J. Gómez-Luna, Y. Guo, S. Brocard, J. Legriel, R. Cimadomo, G. F. Oliveira, G. Singh, and O. Mutlu, "Evaluating machine learning workloads on memory-centric computing systems," in *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2023, pp. 35–49.
- [41] M. He, C. Song, I. Kim, C. Jeong, S. Kim, I. Park, M. Thottethodi, and T. N. Vijaykumar, "Newton: A dram-maker's accelerator-in-memory (aim) architecture for machine learning," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020, pp. 372–385.
- [42] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, "NeuPIMs: NPU-PIM Heterogeneous Acceleration for Batched LLM Inferencing," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 722–737.
- [43] W. Huangfu, X. Li, S. Li, X. Hu, P. Gu, and Y. Xie, "MEDAL: Scalable DIMM based Near Data Processing Accelerator for DNA Seeding Algorithm," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 587–599.
- [44] G. Jonatan, H. Cho, H. Son, X. Wu, N. Livesay, E. Mora, K. Shivdikar, J. L. Abellán, A. Joshi, D. Kaeli, and J. Kim, "Scalability limitations of processing-in-memory using real system evaluations," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 8, no. 1, 2024.
- [45] L. Ke, U. Gupta, B. Y. Cho, D. Brooks, V. Chandra, U. Diril, A. Firoozshahian, K. Hazelwood, B. Jia, H.-H. S. Lee, M. Li, B. Maher, D. Mudigere, M. Naumov, M. Schatz, M. Smelyanskiy, X. Wang, B. Reagen, C.-J. Wu, M. Hempstead, and X. Zhang, "RecNMP: Accelerating Personalized Recommendation with Near-Memory Processing," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 790–803.
- [46] L. Ke, X. Zhang, J. So, J.-G. Lee, S.-H. Kang, S. Lee, S. Han, Y. Cho, J. H. Kim, Y. Kwon, K. Kim, J. Jung, I. Yun, S. J. Park, H. Park, J. Song, J. Cho, K. Sohn, N. S. Kim, and H.-H. S. Lee, "Near-Memory Processing in Action: Accelerating Personalized Recommendation With AxDIMM," *IEEE Micro*, vol. 42, no. 1, pp. 116–127, 2022.
- [47] B. Kim, S. Cha, S. Park, J. Lee, S. Lee, S.-h. Kang, J. So, K. Kim, J. Jung, J.-G. Lee, S. Lee, Y. Paik, H. Kim, J.-S. Kim, W.-J. Lee, Y. Ro, Y. Cho, J. H. Kim, J. Song, J. Yu, S. Lee, J. Cho, and K. Sohn, "The Breakthrough Memory Solutions for Improved Performance on LLM Inference," *IEEE Micro*, vol. 44, no. 3, pp. 40–48, 2024.
- [48] B. J. Kim, T. Jeong, S. Yun, and E.-Y. Chung, "DH-PIM: Maximizing Computing Unit Utilization in Digital PIM by Dual Half Mode Extension," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, pp. 1–1, 2025.
- [49] H. Kim, H. Ye, T. Mudge, R. Dreslinski, and N. Talati, "RecPIM: A PIM-Enabled DRAM-RRAM Hybrid Memory System For Recommendation Models," in *2023 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*, 2023, pp. 1–6.
- [50] Y. Kwon, Y. Lee, and M. Rhu, "Tensordimm: A practical near-memory processing architecture for embeddings and tensor operations in deep learning," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, p. 740–753.
- [51] D. Lee, J. So, M. AHN, J.-G. Lee, J. Kim, J. Cho, R. Oliver, V. C. Thummala, R. s. JV, S. S. Upadhy, M. I. Khan, and J. H. Kim, "Improving in-memory database operations with acceleration dimm (axdim)," in *Proceedings of the 18th International Workshop on Data Management on New Hardware*, 2022.
- [52] J. Lee, I. Hwang, S. Shah, and M. Cho, "FlexReduce: Flexible all-reduce for distributed deep learning on asymmetric network topology," in *2020 57th ACM/IEEE Design Automation Conference (DAC)*, 2020, pp. 1–6.
- [53] S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim, J. Jeon, N. Kim, Y. Kwon, K. Vladimir, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, J. Lee, D. Ko, Y. Jun, K. Cho, I. Kim, C. Song, C. Jeong, D. Kwon, J. Park, J. Chun, and J. Cho, "A 1ynm 1.25v 8gb, 16gb/s/pin gddr6-based accelerator-in-memory supporting Itflops mac operation and various activation functions for deep-learning applications," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65, 2022, pp. 1–3.
- [54] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin, J. Kim, O. Seongil, A. Iyer, D. Wang, K. Sohn, and N. S. Kim, "Hardware architecture and software stack for pim based on commercial dram technology : Industrial product," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 43–56.
- [55] A. Li, S. L. Song, J. Chen, J. Li, X. Liu, N. R. Tallent, and K. J. Barker, "Evaluating modern GPU interconnect: PCIe, NVLink, NV-SLI, NVSwitch and GPUDirect," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 1, pp. 94–110, 2020.
- [56] C. Li, Y. Yin, X. Wu, J. Zhu, Z. Gao, D. Niu, Q. Wu, X. Si, Y. Xie, C. Zhang, and G. Sun, "H2-LLM: Hardware-Dataflow Co-Exploration for Heterogeneous Hybrid-Bonding-based Low-Batch LLM Inference," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 194–210.
- [57] C. Li, Z. Zhou, Y. Wang, F. Yang, T. Cao, M. Yang, Y. Liang, and G. Sun, "PIM-DL: Expanding the applicability of commodity DRAM-PIMs for deep learning via algorithm-system co-optimization," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2024, p. 879–896.
- [58] X. Li, Z. Huang, S. Tao, Z. Huang, C. Zhuang, H. Wang, Y. Li, Y. Qiu, G. Luo, H. Li, H. Shen, M. Chen, D. Bu, W. Zhu, Y. Cai, X. Xiong, Y. Jiang, Y. Heng, P. Zhang, B. Yu, B. Xie, and Y. Bao, "ieda: An open-source infrastructure of eda," in *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2024, pp. 77–82.
- [59] C. Lim, S. Lee, J. Choi, J. Lee, S. Park, H. Kim, J. Lee, and Y. Kim, "Design and analysis of a processing-in-dimm join algorithm: A case study with upmem dimms," *Proc. ACM Manag. Data*, vol. 1, no. 2, 2023.
- [60] C.-C. Liu, C.-F. Wu, and Y. Jin, "Upvss: Jointly managing vector similarity search with near-memory processing systems," in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, 2025, pp. 1–7.
- [61] H. Liu, L. Zheng, Y. Huang, C. Liu, X. Ye, J. Yuan, X. Liao, H. Jin, and J. Xue, "Accelerating personalized recommendation with cross-level near-memory processing," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023.
- [62] L. Liu, S. Zhao, B. Li, H. Ren, Z. Xu, M. Wang, X. Li, Y. Han, and Y. Wang, "Make LLM Inference Affordable to Everyone: Augmenting GPU Memory with NDP-DIMM," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 1751–1765.
- [63] D. Mudigere, Y. Hao, J. Huang, Z. Jia, A. Tulloch, S. Sridharan, X. Liu, M. Ozdal, J. Nie, J. Park, L. Luo, J. A. Yang, L. Gao, D. Ivchenko, A. Basant, Y. Hu, J. Yang, E. K. Ardestani, X. Wang, R. Komuravelli, C.-H. Chu, S. Yilmaz, H. Li, J. Qian, Z. Feng, Y. Ma, J. Yang, E. Wen, H. Li, L. Yang, C. Sun, W. Zhao, D. Melts, K. Dhulipala, K. Kishore, T. Graf, A. Eisenman, K. K. Matam, A. Gangidi, G. J. Chen, M. Krishnan, A. Nayak, K. Nair, B. Muthiah, M. khorashadi, P. Bhattacharya, P. Lapukhov, M. Naumov, A. Mathews, L. Qiao, M. Smelyanskiy, B. Jia, and V. Rao, "Software-hardware co-design for fast and scalable training of deep learning recommendation models," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, p. 993–1011.
- [64] O. Mutlu, "Processing data where it makes sense in modern computing systems: Enabling in-memory computation," in *2018 7th Mediterranean Conference on Embedded Computing (MECO)*, 2018, pp. 8–9.
- [65] O. Mutlu, S. Ghose, J. Gómez-Luna, R. Ausavarungnirun, M. Sadrosadati, and G. F. Oliveira, "A modern primer on processing in memory," 2025. [Online]. Available: <https://arxiv.org/abs/2012.03112>
- [66] L. Nai, R. Hadidi, J. Sim, H. Kim, P. Kumar, and H. Kim, "GraphPIM: Enabling Instruction-Level PIM offloading in graph computing frameworks," in *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2017, pp. 457–468.
- [67] M. Naumov, D. Mudigere, H.-J. M. Shi, J. Huang, N. Sundaraman, J. Park, X. Wang, U. Gupta, C.-J. Wu, A. G. Azzolini, D. Dzulgakov, A. Mallevich, I. Cherniavskii, Y. Lu, R. Krishnamoorthi, A. Yu, V. Kondratenko, S. Pereira, X. Chen, W. Chen, V. Rao, B. Jia, L. Xiong, and M. Smelyanskiy, "Deep learning recommendation model for personalization and recommendation systems," 2019. [Online]. Available: <https://arxiv.org/abs/1906.00091>

- [68] S. U. Noh, J. Hong, C. Lim, S. Park, J. Kim, H. Kim, Y. Kim, and J. Lee, "PID-Comm: A Fast and Flexible Collective Communication Framework for Commodity Processing-in-DIMM Devices," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 245–260.
- [69] S.-S. Park, K. Kim, J. So, J. Jung, J. Lee, K. Woo, N. Kim, Y. Lee, H. Kim, Y. Kwon, J. Kim, J. Lee, Y. Cho, Y. Tai, J. Cho, H. Song, J. H. Ahn, and N. S. Kim, "An LPDDR-based CXL-PNM Platform for TCO-efficient Inference of Transformer-based Large Language Models," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 970–982.
- [70] J. W. Poulton, J. M. Wilson, W. J. Turner, B. Zimmer, X. Chen, S. S. Kudva, S. Song, S. G. Tell, N. Nedovic, W. Zhao, S. R. Sudhakaran, C. T. Gray, and W. J. Dally, "A 1.17-pj/b, 25-gb/s/pin ground-referenced single-ended serial link for off- and on-package communication using a process- and temperature-adaptive voltage regulator," *IEEE Journal of Solid-State Circuits*, vol. 54, no. 1, pp. 43–54, 2019.
- [71] K. Ranganath, A. Abdolrashidi, S. L. Song, and D. Wong, "Speeding up collective communications through Inter-GPU Re-Routing," *IEEE Computer Architecture Letters*, vol. 18, no. 2, pp. 128–131, 2019.
- [72] S. Rashidi, W. Won, S. Srinivasan, S. Sridharan, and T. Krishna, "Themis: a network bandwidth-aware collective scheduling policy for distributed training of DL models," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, p. 581–596.
- [73] C. Renggli, S. Ashkboos, M. Aghagolzadeh, D. Alistarh, and T. Hoefler, "SparCML: High-performance sparse communication for machine learning," in *SC19: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2019, pp. 1–15.
- [74] V. Seshadri, D. Lee, T. Mullins, H. Hassan, A. Boroumand, J. Kim, M. A. Kozuch, O. Mutlu, P. B. Gibbons, and T. C. Mowry, "Ambit: In-memory accelerator for bulk bitwise operations using commodity dram technology," in *2017 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2017, pp. 273–287.
- [75] S. Shi, F. Yang, Z. Li, X. Li, and N. Sun, "Exploring the DIMM PIM architecture for accelerating time series analysis," *IEEE Computer Architecture Letters*, vol. 24, no. 1, pp. 169–172, 2025.
- [76] H. Son, G. Jonatan, X. Wu, H. Cho, K. Shivdikar, J. L. Abellán, A. Joshi, D. Kaeli, and J. Kim, "PIMnet: A domain-specific network for efficient collective communication in scalable PIM," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 1557–1572.
- [77] W. Sun, Z. Li, S. Yin, S. Wei, and L. Liu, "ABC-DIMM: Alleviating the Bottleneck of Communication in DIMM-based Near-Memory Processing with Inter-DIMM Broadcast," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 237–250.
- [78] R. Thakur, R. Rabenseifner, and W. Gropp, "Optimization of collective communication operations in MPICH," *Int. J. High Perform. Comput. Appl.*, vol. 19, no. 1, p. 49–66, 2005.
- [79] B. Tian, Y. Li, L. Jiang, S. Cai, and M. Gao, "NDPBridge: Enabling cross-bank coordination in Near-DRAM-Bank processing architectures," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 628–643.
- [80] J. Wang, M. Ge, B. Ding, Q. Xu, S. Chen, and Y. Kang, "Nicepim: Design space exploration for processing-in-memory dnn accelerators with 3-d stacked-dram," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 5, pp. 1456–1469, 2024.
- [81] C. Wolf, "Yosys open synthesis suite."
- [82] H. Yokoyama and T. Araki, "Efficient distributed machine learning for large-scale models by reducing redundant communication," in *2017 IEEE SmartWorld, Ubiquitous Intelligence & Computing, Advanced & Trusted Computed, Scalable Computing & Communications, Cloud & Big Data Computing, Internet of People and Smart City Innovation (SmartWorld/SCALCOM/UIC/ATC/CBDCOM/IOP/SCI)*, 2017, pp. 1–8.
- [83] F. Zhang, S. Angizi, N. A. Fahmi, W. Zhang, and D. Fan, "PIM-Quantifier: A processing-in-memory platform for mRNA quantification," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, 2021, pp. 43–48.
- [84] M. Zhang, Y. Zhuo, C. Wang, M. Gao, Y. Wu, K. Chen, C. Kozyrakis, and X. Qian, "GraphP: Reducing communication for pim-based graph processing with efficient data partition," in *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2018, pp. 544–557.
- [85] Q. Zhou, C. Chu, N. S. Kumar, P. Kousha, S. M. Ghazimirsaeed, H. Subramoni, and D. K. Panda, "Designing High-Performance MPI libraries with on-the-fly compression for modern GPU clusters," in *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2021, pp. 444–453.
- [86] Z. Zhou, C. Li, F. Yang, and G. Sun, "DIMM-Link: Enabling efficient inter-dimm communication for near-memory processing," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023, pp. 302–316.
- [87] X. Zhu, D. Huang, and Y. Lu, "Enhancing distributed graph matching algorithm with MPI RMA based active messages," in *2023 9th International Conference on Computer and Communications (ICCC)*, 2023, pp. 1952–1961.